04b. Database notes

Status	Draft v1.0. Supporting notes for `04-database-design.md` (the table and index list): design rules, cross-database references, vector design, migration from today's tables, Redis keys, personal data, MVP schema delta
Date	2026-10-20
Depends on	`04-database-design.md`, `architecture.md`

1. Design rules

1. **Two databases, two owners.** `shazlo_core` belongs to the **core service** (Django): identity, discovery, closets, **cart, stock, orders, payments, refunds, returns**, embeddings, and a read copy of the catalog. `shazlo_vendor` belongs to the **vendor-admin service** (FastAPI): vendors, vendor and admin logins, catalog authoring, a read copy of orders, commission history, payouts. Both live on one Postgres server with separate roles; **no service reads or writes the other's database**.
2. **Money, stock and order state are changed only in `shazlo_core`**, inside core transactions. The vendor-admin database never holds a payment or a stock count.
3. **Postgres is the source of truth** in each service. Redis (target) holds only hot copies and short-lived working data (section 5).
4. **UUID primary keys** everywhere; append-only event tables use `bigint` identity keys. Every UUID-keyed table has `created_at` and `updated_at`.
5. **Money is an integer number of paise** (`bigint`), never text or float. Currency is INR.
6. **State lives in a column with a CHECK or choices list**; allowed transitions are enforced in one service function in the owning service, never by direct updates.
7. **Constraints in the database:** unique, foreign key (inside one database), NOT NULL, CHECK (price above zero, stock non-negative, quantity 1 to 20).
8. **Across the two databases there are no foreign keys.** A column that points into the other database is a plain id with no constraint. Consistency comes from the contract (`05` section 8), version numbers and reconciliation (section 2).
9. **Copies are read-only.** `catalog_items` in core and `vendor_orders` in the vendor-admin database are written only by the event receiver of the owning direction.
10. **Snapshots on orders:** an order copies the address, item titles, prices, the catalog versions it was priced at and the commission rate, so later edits never change history.
11. **Every status change is an event row** (who, when, why) in the owning service.
12. Physical table names follow each framework's convention (Django `<app>_<model>`; SQLAlchemy as declared); this document uses short logical names.

2. References that cross the boundary (plain ids, no foreign key)

Column	Points to	Kept consistent by
core `catalog_items.id` (vendor items)	vendor `products.id`	`product.published`, `product.updated`, `product.unpublished` events
core `catalog_items.vendor_id`, `inventory.vendor_id`, `orders.vendor_id`	vendor `vendors.id`	`vendor.status_changed`; vendor ids come from product events
core `inventory.variant_id`, `cart_items.variant_id`, `order_items.variant_id`	vendor `product_variants.id`	The variant ids inside `catalog_items.variants`; core creates an `inventory` row when it first sees a variant

Column	Points to	Kept consistent by
vendor `vendor_orders.order_id`	core `orders.id`	`order.placed`, `order.status_changed`, `payment.*` events and the command responses
vendor `vendor_orders.customer_id`	core `users.id`	`customer.deleted` event anonymises

Reconciliation: each copy carries a `version`. A nightly job in each service lists the other's versions (`GET /internal/v1/versions`) and asks for anything older to be re-sent; a manual `resync` command does the same on demand.

3. Vector design (core only; decision `0003`: pgvector)

- Embeddings are 512 numbers produced by the CLIP service on Modal `[As-is]`; products without one are not eligible for the feed. **Core** requests them after `product.published` and stores them in `product_embeddings`; the vendor-admin service never sees vectors.
- Store them in `product_embeddings.embedding` (`vector(512)`), with an **HNSW index using cosine distance**. Query with the `<=>` operator, filtered by `catalog_items.feed_eligible`, gender, price and category.
- A user's taste lives in `user_taste_clusters.centroid` (`vector(512)`). The update rule is `u = normalize(u + eta * weight * e)`, with eta decreasing as the user swipes more; the maths lives in `feed/taste.py` as a pure function.
- Subtract the catalog's mean embedding before comparing, so dislikes can push away from similar items.
- Filtered nearest-neighbour queries should be tested for recall once the catalog grows; at a few thousand products an exact scan is also acceptable.
- Only `shazlo_core` needs the `pgvector` extension; the Postgres image or host must provide it before the first core migration `[Assumption]` (the `pgvector/pgvector` Docker image does).

4. Migration map from today's tables `[As-is to Target]`

Today everything is in one Django database (`api_*`). The migration splits it across the two databases.

Today (`api_*`)	Becomes	Where	Notes
`api_user`	`users`, `user_profiles`, `user_taste_clusters` (cluster 0), `devices`	core	Keep the UUID. `is_shadow` becomes `is_guest`. Today's rows may store the **phone number in `email`**: split by whether the value contains `@`. `preference_vector` JSON becomes the centroid (skip rows whose length is not 512). `seen_items` is dropped (derived from swipe events). `rewards` becomes `rewards_balance`. `fcm_token` becomes a device row
`api_address`	`addresses`	core	Direct
`api_brand`	`vendors` (status `applied`, or `approved` for the real pilot vendors), `vendor_users` (status `invited`)	**vendor-admin**	A one-time script reads the old table and writes through the vendor-admin service's import command. Password values are discarded (owners get an invite to set a new one); bank fields are not migrated. The old table is then dropped from core
`api_item` (scraped)	`catalog_items` (`source = 'scraped'`), `product_embeddings`	core	One default entry in `variants` per item, price from `price_value` times 100 (parse the text price when `price_value` is empty). `images` JSON becomes the `images` column. Gender: empty string becomes `women`. `embedding` JSON becomes `vector(512)`. Scraped items are link-out only, so they get no `inventory` row

Today (`api_*`)	Becomes	Where	Notes
`api_item` (any real vendor item, if one exists)	`products`, `product_variants`, `product_images` in the vendor-admin database, then published to core, with `inventory` rows created from `stock_quant`	vendor-admin and core	Count first; likely none
`api_itemvariant`	The `variants` column of `catalog_items`	core	Text price becomes integer paise
`api_action`	`swipe_events`	core	`like_status` `"True"` becomes `like`, `"False"` `dislike`, `"2"` `cart`, `"1.5"` `save`; `liked_at` becomes `created_at`
`api_closet` (users and items many-to-many)	`closets`, `closet_members` (earliest member becomes `owner`), `closet_items`	core	
`api_cart.items` (JSON)	`carts`, `cart_items`	core	Map each item to its default variant id
`api_order` and its items	Archived	neither	Checkout was off, so likely few or none; count first and archive instead of migrating if they are test rows

Order of backfill: vendor-admin (categories, vendors, vendor users) and core (users, profiles, addresses) in parallel; then core catalog copy and embeddings, taste clusters, closets, swipe events, carts. Run each as a command that can be repeated safely, compare row counts after each step, and keep the old tables read-only for one release.

5. Redis key layout `[Target; names are proposals]`

The MVP uses no Redis in either service; stock and orders use Postgres rows and transactions.

Service	Instance	Key pattern	Holds	If lost
core	state	`feed:q:{user_id}`	Precomputed next products	Rebuilt from Postgres on the next request
core	state	`feed:taste:{user_id}`	Hot copy of the taste vector	Reloaded from `user_taste_clusters`
core	state	`feed:seen:{user_id}`	Bloom filter of seen products	Rebuilt from `swipe_events`
core	state	`feed:trending`	Sorted set of trending products	Recomputed by a scheduled task
core	state	`feed:swipes`	Stream of swipes, consumer group `inserters`	**The only data at risk**; AOF reduces the window
core	state	`auth:rt:{family_id}`	Refresh-token family (replaces the `refresh_tokens` table)	Users sign in again
core	state	ARQ or Celery queues	Pending jobs (embeddings, timers, event delivery)	Jobs are idempotent; re-queue from source rows and the outbox
core	cache	`cache:product:{id}`, `cache:filters`	Rebuildable data	Recomputed
vendor-admin	state	ARQ queues	Imports, notifications, event delivery	Re-queue from the outbox and job rows

Stock holds are **not** in Redis in this design: `stock_reservations` is a table, so a Redis loss cannot lose or double a reservation.

6. PII, retention and deletion `[Assumption: confirm with counsel and the accountant]`

Data	Where	Class	Rule
`users` (email, phone, name, birth date), `addresses`, `devices`	core	Personal	Deleted or anonymised on account deletion
`swipe_events`, `user_taste_clusters`, `closets` data	core	Pseudonymous	Deleted with the account
`orders`, `payments`, `refunds` (customer name, phone, address snapshot)	core	Personal and financial	Kept for the legally required period, then anonymised; **personal fields blanked on account deletion**, money rows kept

Data	Where	Class	Rule
`vendor_orders` (customer name, phone, address)	vendor-admin	Personal copy	Blanked when the `customer.deleted` event arrives; the rest kept for statements
`vendor_users`, `vendors`	vendor-admin	Business contact data	Kept while the vendor relationship exists
`audit_log`, `notification_log`, `vendor_notification_log`	both	Operational	No message bodies; keep for a fixed period

Deleting a customer is two steps across the services: core deletes its data and emits `customer.deleted`; the vendor-admin service anonymises its copy. If the event is delayed, deletion in core has already happened.

7. MVP schema delta `[MVP]`

The MVP does **not** create the whole target schema.

Core (today's `api` tables, Django migrations) gains the commerce tables and changes minimally elsewhere (exact field names in `06-mvp-slice-and-migration.md`):

- New: `Inventory`, `StockReservation`, `OrderItem` (replacing the many-to-many), `OrderStatusEvent`, `Payment` (provider `none` at launch), `OutboxEvent`, `InboxEvent`, `IdempotencyKey`, `RefreshToken`. After the core loop works: `Refund`, `Delivery`, `Return`.
- `Order` is reshaped (vendor id, status, payment method and status, commission snapshot, price versions, timers, idempotency key).
- `Item` and `ItemVariant` become the catalog copy: they gain `source`, `status`, `vendor_id`, `vendor_name`, `commission_bps`, `version`, `feed_eligible`, fabric, returnable fields and a required numeric `price_value`; variant price becomes integer paise. Vendor products arrive by event; scraped products stay.
- `Action.like_status` values are normalised to the swipe-action enum. `Brand` is retired from core after the one-time copy to the vendor-admin database (section 4).

Vendor-admin (`shazlo_vendor`, new, Alembic migrations) creates only what the core loop needs: `vendors`, `commission_rates`, `vendor_users`, `admin_users`, `refresh_tokens`, `vendor_policies`, `categories`, `products`, `product_variants`, `product_images`, `product_status_events`, `vendor_orders`, `vendor_order_items`, `outbox_events`, `inbox_events`, `idempotency_keys`, `vendor_notification_log`, `audit_log`. Added after the core loop works: `vendor_locations`, `vendor_status_events`, `import_jobs`, `vendor_payouts`, `payout_lines`.

These changes are chosen so the migration in section 4 stays mechanical.

8. Open items

Whether `gstin` is required for vendors; guest cleanup age; how long order data is kept; whether `feed_impressions` is sampled; the unpaid online-order window (about 15 minutes); the stock reservation expiry for cash orders (Q40); payout eligibility rules and how online money reaches vendors (Q37); price staleness window (Q39). All tracked in `open-questions.md`. Decided and no longer open: return window (7 days), commission (15%), confirmation window (4 business hours), commerce in core, stock owned by core, two databases.